

1 First multiplicative calculus rules

Source: PrimeTensor/Analysis/Rules.lean

What this file establishes

PrimeTensor/Analysis/Derivative.lean defined the multiplicative derivative and computed it only where the first-order model happens to be exact. This file proves the first two rules that combine derivatives:

$$D_{fg} = D_f \cdot D_g, \quad D_{1/f} = D_f^{-1},$$

with the tangent maps multiplied and inverted pointwise. Both are exactly what the module header promises — the product consumes one intrinsic scale level, inversion consumes none — and the level accounting appears in a new place, which Design note 1.8 is about.

The striking feature is the shape of the product rule. There is no Leibniz cross term: the model for fg depends on D_f and D_g and on nothing else — not on $f(x)$, not on $g(x)$. Design note 1.13 explains why that is right and not a simplification.

The file is built in four layers, each a product-and-inverse pair: carrier algebra on MulReal, operations on tangent maps, closure properties of negligibility and first-order equivalence, and the two rules themselves.

1.1 Carrier algebra, one layer up

Lemma 1.1 (MulReal.inv_mul_pair and mul_four_shuffle). $(ab)^{-1} = a^{-1}b^{-1}$ and $(ab)(cd) = (ac)(bd)$ on MulReal.

```
theorem mul_four_shuffle (a b c d : MulReal) :  
  (a * b) * (c * d) = (a * c) * (b * d) := by  
  calc  
    (a * b) * (c * d)  
      = a * (b * (c * d)) := mul_assoc a b (c * d)  
    _ = a * ((b * c) * d) := by  
      rw [← mul_assoc b c d]  
    _ = a * ((c * b) * d) := by  
      rw [mul_comm b c]  
    _ = a * (c * (b * d)) := by  
      rw [mul_assoc c b d]  
    _ = (a * c) * (b * d) := (mul_assoc a c (b * d)).symm
```

Proof. The first descends from its MulRat counterpart in PrimeTensor/Algebra.lean by the standard route: induct to representatives, apply Quotient.sound, apply of_pointwise, and close termwise. The second is the five-step regrouping, written out for the same reason it was written out one layer down. $\square$

Lemma 1.2 (MulReal.ratio_mul_pair, ratio_inv_pair and ratio_reverse).

$$\text{ratio}(ac, bd) = \text{ratio}(a, b) \text{ratio}(c, d), \quad \text{ratio}(a^{-1}, b^{-1}) = \text{ratio}(b, a), \quad \text{ratio}(b, a) = \text{ratio}(a, b)^{-1}.$$

Proof. The first two are the arguments of PrimeTensor/Algebra.lean repeated with Lemma 1.1 in place of its MulRat version. For the third, $\text{ratio}(b, a) = ba^{-1}$ and $\text{ratio}(a, b)^{-1} = (ab^{-1})^{-1} = a^{-1}b$, and the two agree by commutativity. $\square$

Remark 1.3 (Where the mixin could have been used and was not). Every statement in Lemmas 1.1 and 1.2 is a consequence of the six laws of `IsMulCarrier` alone, and both `MulRat` and `MulReal` are instances. They are nonetheless proved twice — once in `PrimeTensor/Algebra.lean` for the finite carrier and once here for the completed one — and the completed proof of `inv_mul_pair` even descends through representatives rather than using the laws.

The obstacle is not the laws but the notation: `ratio` is defined separately on `MulRat` and on `MulReal`, and not once for a general carrier, so a generic statement has nothing to be about. This is the first concrete place where the payoff of `PrimeTensor/Structure.lean`, described there as entirely deferred, could have been collected and is not.

1.2 Operations on tangent maps

Definition 1.4 (`MulTangentMap.mul` and `MulTangentMap.inv`). $(D \cdot E)(h) \equiv D(h) \cdot E(h)$ and $D^{-1}(h) \equiv D(h)^{-1}$, each carrying proofs that the result is again a tangent map.

```
def mul (D E : MulTangentMap) : MulTangentMap where
  toFun := fun h => D h * E h
  map_one := by
    rw [D.map_one, E.map_one]
    exact MulReal.one_mul 1
  map_mul := by
    intro a b
    rw [D.map_mul, E.map_mul]
    exact MulReal.mul_four_shuffle (D a) (D b) (E a) (E b)
  map_inv := by
    intro a
    rw [D.map_inv, E.map_inv]
    symm
    exact MulReal.inv_mul_pair (D a) (E a)
```

Verification for the product. The pivot goes to $1 \cdot 1 = 1$. Multiplicativity is $D(ab)E(ab) = D(a)D(b)E(a)E(b) = (D(a)E(a))(D(b)E(b))$, the middle step being exactly the four-factor regrouping of Lemma 1.1. Preservation of inverses is $D(a)^{-1}E(a)^{-1} = (D(a)E(a))^{-1}$, the same lemma read backwards. The inverse tangent map is verified the same way, with `inv_one` for the pivot. $\square$

Remark 1.5 (A group of tangent maps, not declared). These operations make the tangent maps a commutative group with trivial as unit — they are the endomorphisms of the carrier under pointwise multiplication. Nothing in the file says so: no `One`, `Mul` or `Inv` instance is declared for `MulTangentMap`, and no law is stated for these two definitions. Even $D \cdot \text{trivial} = D$ would need structure extensionality, which is not invoked anywhere. The operations are here to be applied, not to be reasoned about.

1.3 Negligibility is closed under the operations

Theorem 1.6 (`negligible_mul`). If e_1 and e_2 are each negligible relative to a common base, so is $n \mapsto e_1(n) e_2(n)$.

```
theorem negligible_mul
  {e₁ e₂ base : MulReal.Seq}
```

```

(h₁ : NegligibleRelative e₁ base)
(h₂ : NegligibleRelative e₂ base) :
NegligibleRelative (fun n => e₁ n * e₂ n) base := by
intro gain
obtain (aAnchor, ha) := h₁ (.succ gain)
obtain (bAnchor, hb) := h₂ (.succ gain)

```

Proof. Fix a gain g . Ask each hypothesis not for g but for $\text{succ } g$, and take join of the two anchors. For n beyond it and any level ℓ achieved by the base at n , both errors are near the pivot at level $\text{advance}(\ell, \text{succ } g)$, which by advance_succ is $\text{succ advance}(\ell, g)$. So scaleNear_mul of `PrimeTensor/Analysis/Near/Laws.lean` applies and puts the product near $1 \cdot 1$ at level $\text{advance}(\ell, g)$; cancelling the unit gives the claim. $\square$

Theorem 1.7 (`negligible_inv`). *If e is negligible relative to a base, so is $n \mapsto e(n)^{-1}$, at the same gain.*

Proof. Keep the gain, keep the anchor, and apply scaleNear_inv . $\square$

Design note 1.8 (The level cost, transposed into the rate). Everywhere until now the one-level cost of multiplication was paid in the *level*: two hypotheses at $\text{succ } \ell$ gave a conclusion at ℓ . Here it is paid in the *gain* instead: two hypotheses at $\text{succ } g$ give a conclusion at g , and the level ℓ is carried through untouched. The mechanism is advance_succ , the observation that advancing by $\text{succ } g$ is one succ applied to advancing by g — so a request one gain deeper delivers a level one succ finer, which is precisely the currency scaleNear_mul spends.

In the logarithmic chart of `PrimeTensor/Analysis/Derivative.lean` the accounting is the familiar one. Negligibility at gain g says $|\log e| \lesssim 2^{-|g|} |\log \text{base}|$. If both errors satisfy this at gain $\text{succ } g$, their product satisfies

$$|\log e_1 + \log e_2| \leq 2 \cdot 2^{-(|g|+1)} |\log \text{base}| = 2^{-|g|} |\log \text{base}|,$$

so the factor of two from adding two errors is exactly the one refinement level. Inversion negates the logarithm and changes no magnitude, which is why it costs nothing.

Corollary 1.9 (`firstOrder_mul` and `firstOrder_inv`). *First-order equivalence relative to a fixed base is preserved by pointwise multiplication and by pointwise inversion of both germs.*

Proof. For the product, the error germ of the products is $\text{ratio}(a_1 a_2, m_1 m_2)$, which by Lemma 1.2 is the product of the two error germs, so Theorem 1.6 applies. For the inverse, the error germ is $\text{ratio}(a^{-1}, m^{-1})$, which the same lemma rewrites first as $\text{ratio}(m, a)$ and then as $\text{ratio}(a, m)^{-1}$ — the inverse of the original error germ — so Theorem 1.7 applies. $\square$

1.4 Responses factor exactly

Lemma 1.10 (`response_mul` and `response_inv`). *For every f, g, x, h and n ,*

$$\begin{aligned} \text{response}(fg, x, h)_n &= \text{response}(f, x, h)_n \cdot \text{response}(g, x, h)_n, \\ \text{response}(f^{-1}, x, h)_n &= \text{response}(f, x, h)_n^{-1}. \end{aligned}$$

```

theorem response_mul
  (f g : MulReal → MulReal) (x : MulReal) (h : MulReal.Seq) :

```

```

    ∀ n : Depth,
      response (fun y => f y * g y) x h n =
        response f x h n * response g x h n := by
intro n
unfold response
exact MulReal.ratio_mul_pair
  (f (x * h n)) (f x)
  (g (x * h n)) (g x)

```

Proof. Unfold the response to a ratio of values and apply Lemma 1.2 in the two forms just used. $\square$

These are identities, with no anchor, no level and no hypothesis about h . They are the reason the two rules below cost nothing beyond what Corollary 1.9 already paid.

1.5 The two rules

Theorem 1.11 (`hasMulDerivativeAt_mul`). *If f has derivative D_f at x and g has derivative D_g at x , then $y \mapsto f(y)g(y)$ has derivative $D_f \cdot D_g$ at x .*

Proof. Fix a perturbation h approaching the pivot. Both hypotheses apply to it, giving first-order equivalences of the two responses with the two models relative to h , and Corollary 1.9 multiplies them. It remains only to see that the resulting statement is the required one, which is two rewrites: the actual germ is the response of the product, equal by Lemma 1.10 to the product of the responses, and the model germ is $(D_f \cdot D_g)(h_n)$, equal by `mul_apply` to $D_f(h_n)D_g(h_n)$. $\square$

Theorem 1.12 (`hasMulDerivativeAt_inv`). *If f has derivative D at x , then $y \mapsto f(y)^{-1}$ has derivative D^{-1} at x .*

Proof. The same three steps with the inverse halves of Corollary 1.9 and Lemma 1.10. $\square$

Design note 1.13 (No Leibniz term, and why that is correct). The additive product rule is $(fg)' = f'g + fg'$: the model for the product depends on the *values* of f and g at the point, not only on their derivatives. Theorem 1.11 has no such dependence. The model for fg is $D_f \cdot D_g$, and $f(x)$ and $g(x)$ appear nowhere.

The reason is that the multiplicative derivative is the logarithmic derivative in disguise. In the chart of `PrimeTensor/Analysis/Derivative.lean`, D models the increment of $\log \circ f$, so what the theorem asserts is

$$(\log fg)' = (\log f)' + (\log g)',$$

which is not the Leibniz rule but the additivity of differentiation, applied after $\log$ has already turned the product into a sum. Equivalently, in ordinary notation the logarithmic derivative satisfies $(fg)'/(fg) = f'/f + g'/g$: the Leibniz cross terms are exactly what the division by fg cancels. The multiplicative language never writes them down because it never forms the product $f'g$ in the first place.

Theorem 1.12 reads the same way, $(1/f)'/(1/f) = -f'/f$, and its cost-free status has the structural explanation given in `PrimeTensor/Algebra.lean`: the additive quotient rule needs $f \neq 0$ and divides by f^2 , while here there is no zero in the carrier and inversion is an isometry. What is a side condition there is absent here.

Remark 1.14 (Still no genuinely approximate example). `PrimeTensor/Analysis/Derivative.lean` computed two derivatives, both with exact models. The rules proved here combine derivatives that are already known, so applying them to those two examples again produces only exact cases — for instance $y \mapsto y \cdot c$ with model identity, since `identity · trivial` acts as identity does. No function whose model is inexact is exhibited in this file either, so the negligibility machinery, although now equipped with closure properties, has still never been used with a nontrivial error.

Remark 1.15 (What is not proved). There is no quotient rule stated, although $y \mapsto f(y)g(y)^{-1}$ follows immediately from the two theorems; no chain rule, which would need a notion of composing perturbations that no file has yet defined; and still no uniqueness of derivatives. `NegligibleRelative` is shown closed under products and inverses but is not shown transitive, nor related to `ConvergesTo`, and nothing connects having a derivative to any continuity property.

Summary of the correspondence

Lean declaration	Kind	Here
<code>Depth.advance_succ</code>	simp thm	Thm. 1.6
<code>MulReal.inv_mul_pair</code>	theorem	Lem. 1.1
<code>MulReal.mul_four_shuffle</code>	theorem	Lem. 1.1
<code>MulReal.ratio_mul_pair</code>	theorem	Lem. 1.2
<code>MulReal.ratio_inv_pair</code>	theorem	Lem. 1.2
<code>MulReal.ratio_reverse</code>	theorem	Lem. 1.2
<code>MulTangentMap.mul</code>	definition	Def. 1.4
<code>MulTangentMap.inv</code>	definition	Def. 1.4
<code>MulTangentMap.mul_apply</code>	simp thm	Def. 1.4
<code>MulTangentMap.inv_apply</code>	simp thm	Def. 1.4
<code>negligible_mul</code>	theorem	Thm. 1.6
<code>negligible_inv</code>	theorem	Thm. 1.7
<code>firstOrder_mul</code>	theorem	Cor. 1.9
<code>firstOrder_inv</code>	theorem	Cor. 1.9
<code>response_mul</code>	theorem	Lem. 1.10
<code>response_inv</code>	theorem	Lem. 1.10
<code>hasMulDerivativeAt_mul</code>	theorem	Thm. 1.11
<code>hasMulDerivativeAt_inv</code>	theorem	Thm. 1.12

Every declaration of `PrimeTensor/Analysis/Rules.lean` appears above. The file contains no sorry, no axiom, and no unproved named proposition. Design notes 1.8 and 1.13 and Remarks 1.3, 1.5, 1.14 and 1.15 are stated for the reader and are not declarations of the file; the logarithmic readings in the two design notes are not formalised.